Enterprise-Scale Apex Across Multiple Packages

Contracts, configuration and extensibility

FFLib Series · Session 006

John M. Daniel

Senior Director, Digital Platforms · Steampunk, Inc.



About me

Senior Director of Digital Platforms, Steampunk, Inc. | Apex Enterprise Patterns Maintainer & Architect

- **Apex Enterprise Patterns (fflib) maintainer & architect** — steward of `fflib-apex-common`, `fflib-apex-mocks`, `force-di`, and `AT4DX` at github.com/apex-enterprise-patterns
- **Open source education** — promoting awareness and adoption of Domain, Service, and Selector patterns for enterprise Salesforce development
- **Senior Director, Digital Platforms** — leading Salesforce platform engineering standards and delivery at Steampunk, Inc.



When I am not working on Apex architecture, I am probably driving my Jeep on the beach.

FFLib Series

Promises we made

Andy, Session 2: metadata-driven dependency injection, and interfaces to resolve Services	→ slide **19, 24**
Session 4: AT4DX adds fields to a Selector from another package (field sets)	→ slide **30**
Session 4: `@NamespaceAccessible`	→ slide **16**
Session 5: the `WDGT_` prefix, "a namespace inside one package"	→ slide **16**
Session 5: "AT4DX keeps a factory, on purpose"	→ slide **23**

One package so far

Today the same layers split across packages

- **Sessions 1 to 5 lived in one package:** `force-app` . Every class could see every other class
- **Service, Domain, Selector and Unit of Work** don't change today. Where they live does
- **Session 5's seams** — `newInstance()` , constructors, mocks — are what make the split possible

> Same patterns. Different boxes. The boxes are packages.

Today

TODAY	
Meridian: one org, four divisions	☐
The package layers	☐
Cross-package architecture	☐
Shared contracts and interfaces	☐
Configuration-driven design	☐
Extensibility patterns	☐

Meet Meridian Corporation

An industrial supplier with robot-run warehouses, all on one Salesforce org

Sales

Runs the CRM: opportunities and quotes

Built with an outside consultancy

Service

Cases, complaints and returns

Built by a second vendor

Marketing

Campaigns and customer engagement

A small in-house team

Meridian Fulfillment

The Warehouse Operations app: North Hub and its robots

The app from Sessions 2 to 5

For years it worked, because nobody's code touched anyone else's.

Then the divisions need each other

And the org starts turning into Happy Soup

Sales → Fulfillment

A closed deal should create a fulfillment order

Service → Fulfillment

Tell a customer where their order is

Marketing → everyone

Its fields on every Account query

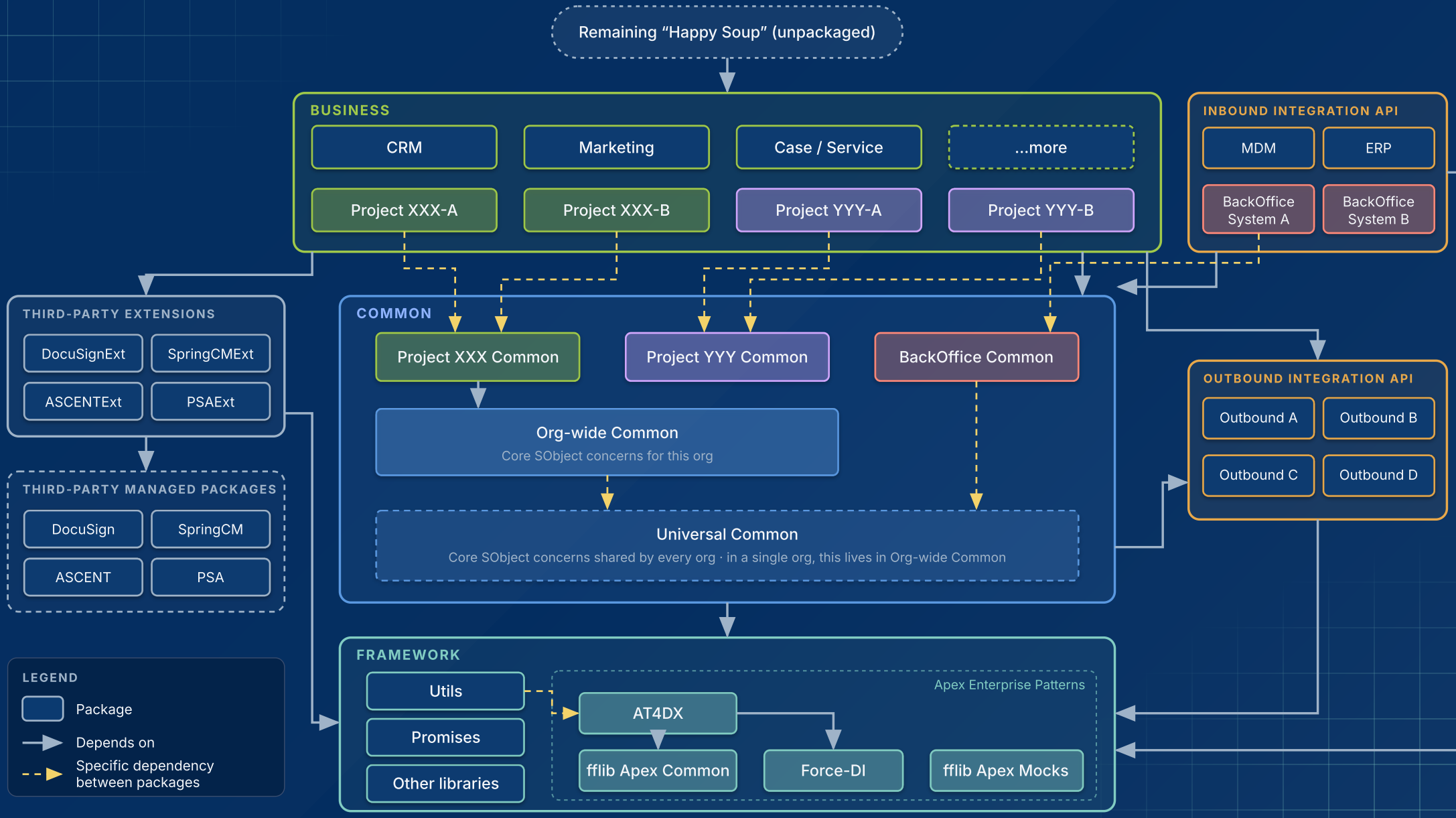
⚠️ **Four teams edit the same Account trigger and the same Account Selector.**
⚠️ **Every release needs everyone.** One vendor's change breaks another vendor's tests.

Leadership's question

> *How do the divisions work together without wiring themselves so tightly that every change ripples through all of them?*

The question from today's abstract

The package layers



What lives in each layer

Top to bottom

Layer	What lives here	Rule
Remaining "Happy Soup"	Metadata not yet packaged	Shrinks over time; depends on everything below it
Business	One package per division or project: CRM, Service, Marketing, project packages	Owns its own objects, fields and processes. Never depends on another business package
Project Common	Shared by one project's business packages	Exists only when two or more business packages need the same thing
Org-wide Common	Core SObject concerns for this org	Owns the shared objects and their one Selector and one Domain
Universal Common	Core SObject concerns common to all of the company's orgs	Multi-org only. In a single org, its contents live in Org-wide Common
Framework	fflib Apex Common and Apex Mocks, force-di, AT4DX, utils	No company schema at all

Beside the stack: integration API packages on common, and third-party extension packages on managed packages.

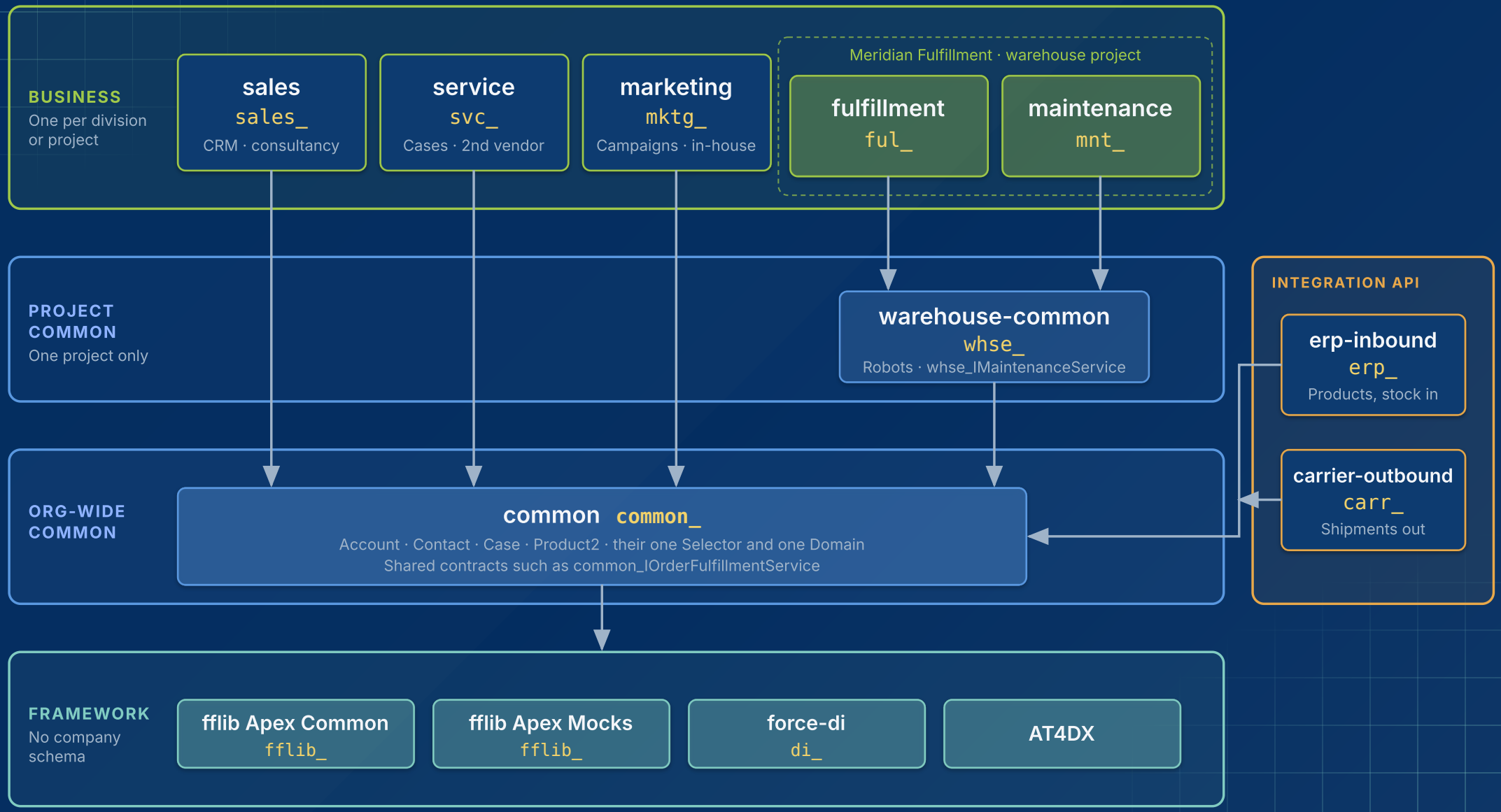
Three rules

1. **Dependencies only point down**
   A lower layer never names anything in a higher one

2. **Each object has one owner**
   The lowest layer that needs it owns it. Higher layers extend it through injection, not by editing it

3. **Push shared things down only as far as needed**
   Two business packages share something? It goes in their Project Common, not Org-wide Common

Meridian on the map



One org, or many

Meridian is a single-org story. The pattern doesn't change with more orgs

- **Single org:** anything that would live in Universal Common goes into that org's Org-wide Common package
- **Several orgs:** it moves down into a Universal Common package that every org installs
- **Everything above it is the same:** business packages, Project Common, contracts and bindings

> Start in one org. If a second org arrives, move shared pieces down a layer.

1 · Cross-package architecture

FFLib's layers already give you the boundaries. Packages make the platform enforce them

Why split at all

- Teams ship on different schedules
- The org is too big for one deploy
- Some features belong to one division or region

What unlocked packages give you

- Each team gets its own version, tests and release
- The platform checks the dependencies
- Ownership is visible, not tribal knowledge

Where FFLib lands

- **Framework:** fflib, force-di, AT4DX
- **Common:** shared objects, their Selector and Domain, contracts
- **Business:** its own Domains, Selectors, Services, registered through custom metadata

Dependencies only point down

The rule that forces everything else today

- `common` **can never name a class in** `sales` **or** `fulfillment` **.** It doesn't compile, because it can't see them
- `sales` **can't name a class in** `fulfillment` **either.** Business packages never depend on each other
- **So when a lower package needs work done above it, it can't call it directly.** It has to ask something for it

> If you can't call it, you ask for it. Topics 2 to 4 are the ways to ask.

Package prefixes, not namespaces

Framed for an enterprise: 2GP unlocked packages, no namespace

- **Each package gets a prefix:** 3 to 8 characters, an underscore, then the element's proper name
- `sales_OpportunitiesService`, `svc_SLATier__c`, `whse_Robot__c`: the prefix says which package owns it
- **Session 5's `WDGT_` prefix was a package prefix inside one package.** Today every package gets one

> Namespaces and `@NamespaceAccessible` are for ISVs building managed packages. The AEP frameworks support that too; it's not today's case.

Naming conventions

"Let the metadata's API name tell you where it belongs"

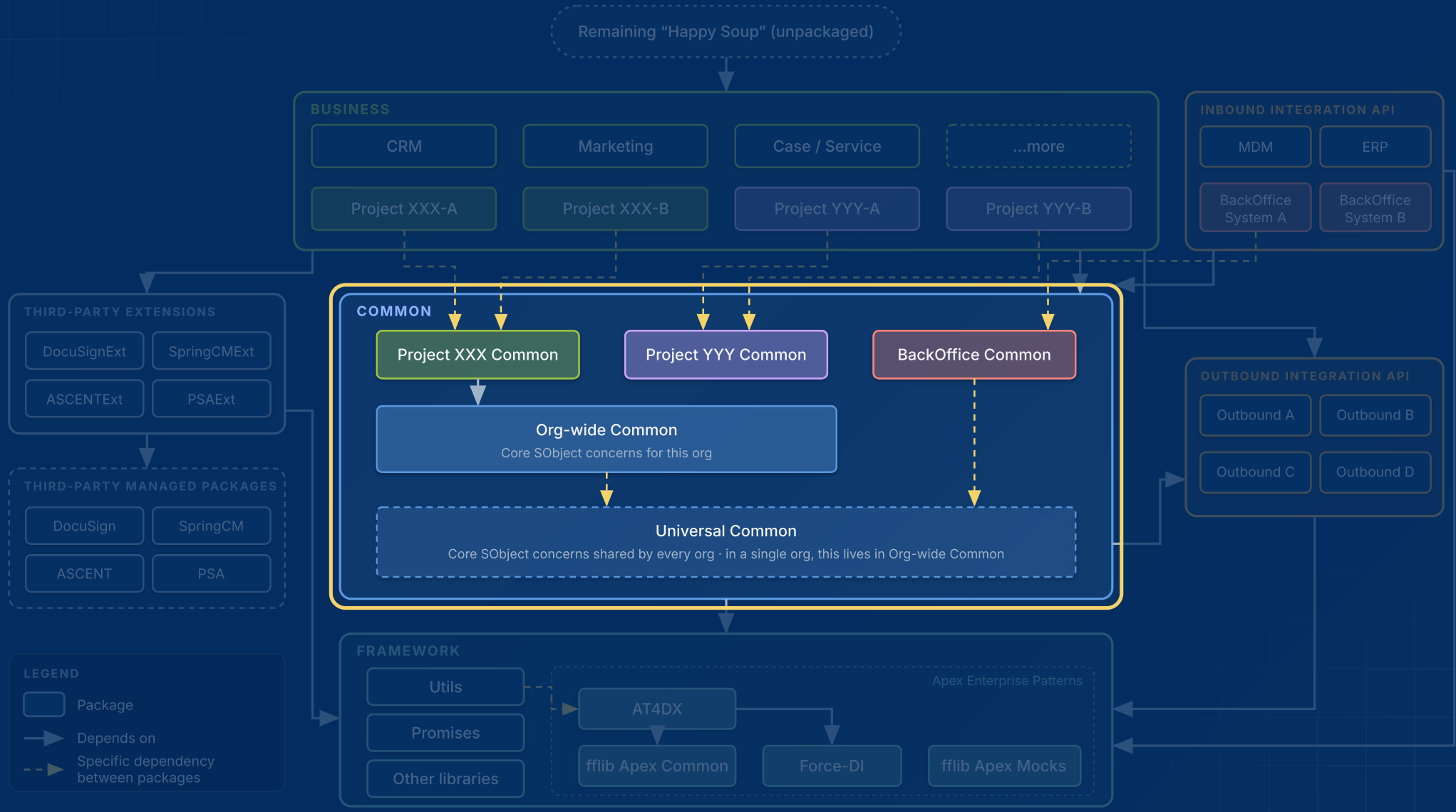
A name has two parts: the **prefix** says which package owns it, and the **suffix** says what kind of artifact it is.

Name	Owner	Artifact
`sales_AccountsSelector`	sales	Selector
`whse_IMaintenanceService`	warehouse-common	Service contract
`mnt_MaintenanceServiceImpl`	maintenance	Service implementation
`svc_CaseController`	service	Controller
`ful_FulfillmentRestAPI`	fulfillment	REST API
`mktg_EngagementScore__c`	marketing	Custom field on Account

> Pays off in code review, in audits, and while code is still moving out of the Happy Soup.

2 · Shared contracts and interfaces

Contracts live in a common layer, so business packages talk without depending on each other



Interfaces return, on purpose

Session 5 saved this case for today

- **Session 5:** use interfaces "only where implementations genuinely vary"
- **Across packages, they always vary:** the caller can't see the implementation at all
- **The lower package publishes the interface.** The higher package supplies the implementation
- **Each side compiles against the contract,** never against the other side

> Andy, Session 2: interfaces are how Services get resolved. Here's where it matters.

What goes in a contract

Small and stable

Put in

- A Service interface, such as `whse_IMaintenanceService`
- The records or simple data classes it passes
- Any exceptions it throws

Leave out

- Domains and Selectors, unless another package really needs them
- Anything only one package uses

Keep it stable

- Every change moves every package that uses it
- Add methods; avoid changing or removing old ones
- Put it in the lowest package both sides depend on, never in the implementing package

Fulfillment calls Maintenance

Inside a package, call directly. Across packages, go through a contract in a lower layer

FulfillmentService.cls · Sessions 2 to 5

```
// One package: it can see MaintenanceService
private final MaintenanceService maintenanceSvc;

public static FulfillmentService newInstance() {
  return new FulfillmentService(
    // ...selectors
    MaintenanceService.newInstance()
  );
}

// Hand due robots to maintenance
maintenanceSvc.scheduleService(dueRobots, uow);
```

ful_FulfillmentService.cls · today

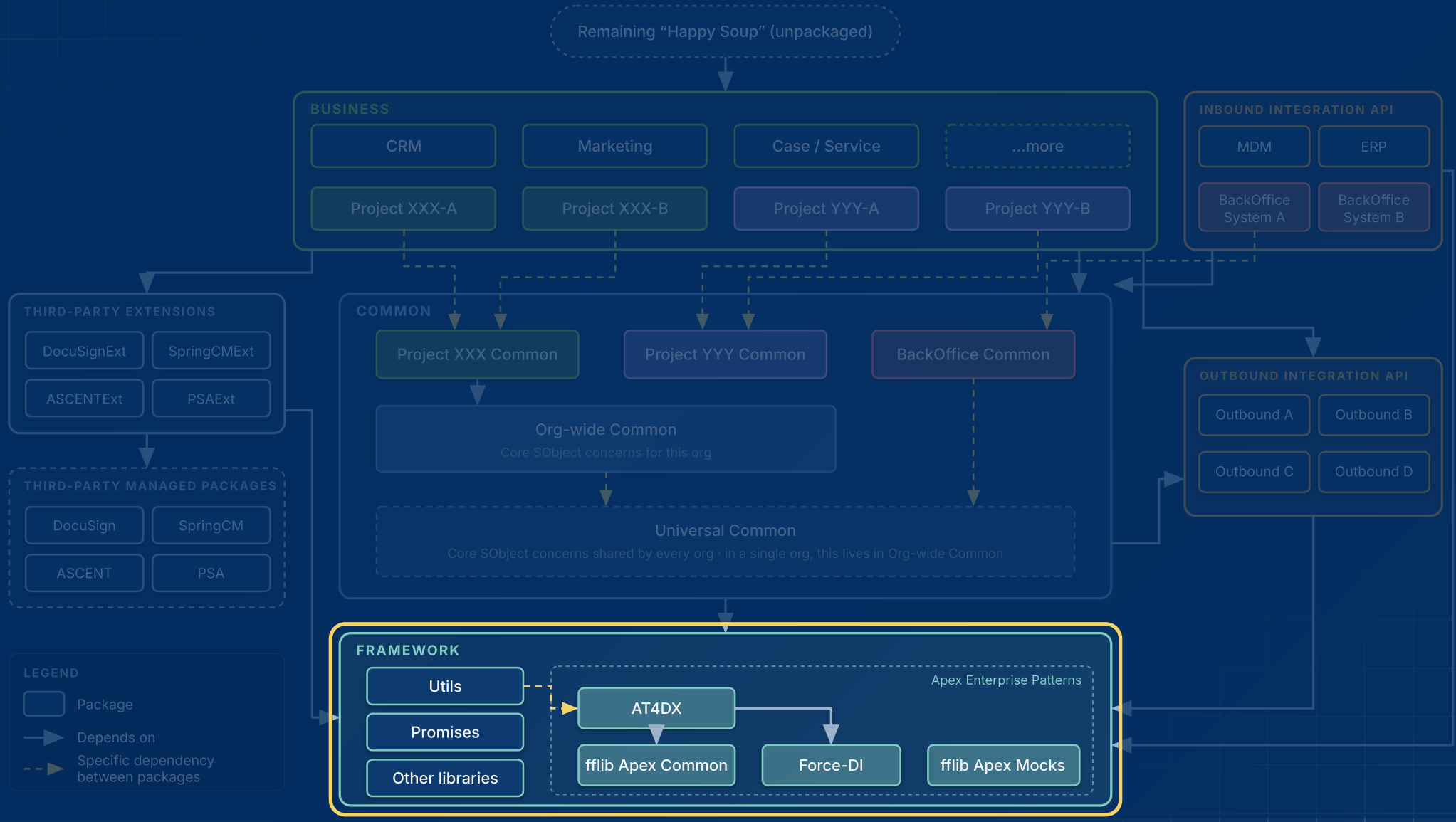
```
// Can't see maintenance: use the contract
private final whse_IMaintenanceService maintenanceSvc;

public static ful_FulfillmentService newInstance() {
  return new ful_FulfillmentService(
    // ...selectors
    (whse_IMaintenanceService) Application.Service
      .newInstance(whse_IMaintenanceService.class)
  );
}

// Same call, through the contract
maintenanceSvc.scheduleService(dueRobots, uow);
```

3 · Configuration-driven design

How a lower layer hands out implementations that live in a higher layer



Why a factory, across packages

"AT4DX keeps a factory, on purpose"

- **Session 5:** `newInstance()` plus constructors. Inside one package, that's enough
- **Across packages, the caller can't write** `new MaintenanceService()` **:** it can't see that class
- **So it asks the factory for** `whse_IMaintenanceService` **,** and the factory looks up the binding
- **There's no app-specific** `Application` **class to edit:** AT4DX's `Application` lives in the framework and reads bindings from every installed package

> The caller asks for an interface. Custom metadata decides the class. Tests still `setMock()` .

Four factories, four bindings

Bindings live in custom metadata, and deploy with the package that owns the implementation

Factory	Custom metadata	Key fields
`Application.Selector`	`ApplicationFactory_SelectorBinding__mdt`	`BindingSObject__c`, `To__c`, `Priority__c`
`Application.Domain`	`ApplicationFactory_DomainBinding__mdt`	`BindingSObject__c`, `To__c` (a Domain's constructor class)
`Application.Service`	`ApplicationFactory_ServiceBinding__mdt`	`BindingInterface__c`, `To__c`, `Priority__c`
`Application.UnitOfWork`	`ApplicationFactory_UnitOfWorkBinding__mdt`	`BindingSObject__c`, `BindingSequence__c`

Installing a package adds its bindings. Nobody edits common. Under the covers it's force-di: metadata-driven dependency injection.

A binding record

Story: Service asks where an order is

`common` publishes `common_IOrderFulfillmentService`. A Case page in `service` calls it. `fulfillment` implements it and ships this record:

ApplicationFactory_ServiceBinding__mdt	ful_OrderFulfillmentService
BindingInterface__c	`common_IOrderFulfillmentService`
To__c	`ful_OrderFulfillmentServiceImpl`
Priority__c	1

> Service never learns that the fulfillment package exists. It only knows the contract in common.

Asking the factory

The same Service triad as the AT4DX samples, with package prefixes

common_OrderFulfillmentService.cls · common

```
1  public inherited sharing class common_OrderFulfillmentService {
2    private static common_IOrderFulfillmentService service() {
3      return (common_IOrderFulfillmentService)
4        Application.Service.newInstance(common_IOrderFulfillmentService.class);
5    }
6
7    public static List<common_OrderStatus> getStatusForAccounts(Set<Id> accountIds) {
8      return service().getStatusForAccounts(accountIds);
9    }
10 }
11
12 // In a test, the mock beats any binding (same as Session 5)
13 Application.Service.setMock(common_IOrderFulfillmentService.class, mockService);
```

Before fulfillment is installed, nothing answers. After install, its binding answers. No change to service or common.

Overrides, and a caution

Configuration is powerful, and it hides the call graph

Overrides without forks

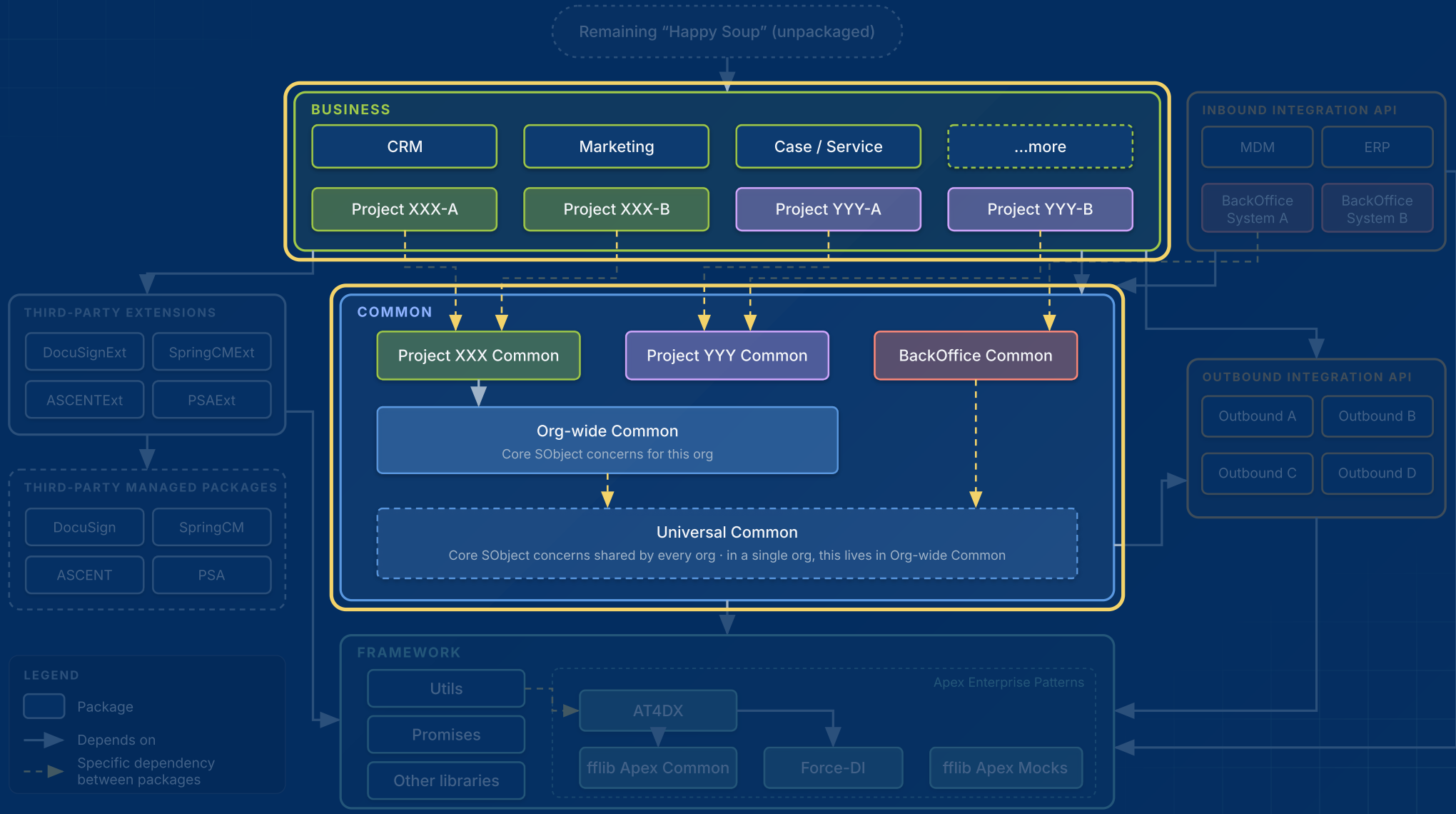
- A later package, or a subscriber org, supplies a different binding for the same interface
- `Priority__c` decides which one wins
- A regional package can replace one Service without touching the original

Keep it traceable

- Name bindings clearly, with the package prefix
- Keep one binding per interface unless you mean to override
- Add a test that checks every binding resolves

4 · Extensibility patterns

How a higher layer adds fields, queries and logic to objects a lower layer owns



Five AT4DX techniques

Each FFLib layer has an extension point

Technique	Extension point	What a higher-layer package adds
Application factory injection	`ApplicationFactory_*Binding__mdt`	Its own Selector, Domain or Service, or an override of one
Selector field injection	`SelectorConfig_FieldSetInclusion__mdt`	Its own fields on another package's queries, through a field set
Selector method injection	`selectInjection(...)` on the shared Selector	Its own query against an object a lower layer owns
Domain process injection	`DomainProcessBinding__mdt`, plus criteria and action classes	Logic inside another package's Domain, in a set order
Platform event distribution	`PlatformEvents_Subscription__mdt`, plus an `IEventsConsumer`	A subscriber to another package's events

Selector field injection

Story: Marketing adds fields to Account

common owns

`Account` and `common_AccountsSelector`, which extends `ApplicationSObjectSelector` with `includeFieldSetFields` on

marketing ships

- `mktg_Slogan__c`, `mktg_EngagementScore__c`
- A field set: `mktg_AccountFields`
- A `SelectorConfig_FieldSetInclusion__mdt` record: `BindingSObject__c` = Account, `FieldsetName__c` = `mktg_AccountFields`

Result

Every Account query through the Selector comes back with marketing's fields

common never changes

"The core package never learns that the marketing package exists, yet Account queries come back with the marketing fields."

Selector method injection

Story: Service needs accounts with more than five open complaint Cases

svc_AccountsWithOpenComplaintsSelectorMethod.cls · service

```
1  public inherited sharing class svc_AccountsWithOpenComplaintsSelectorMethod
2    extends AbstractSelectorMethodInjectable
3    implements ISelectorMethodInjectable {
4
5    public List<SObject> selectQuery() {
6      Parameters params = (Parameters) getParams();
7      Integer minimum = params.minimumOpenComplaints;
8      // newQueryFactory() is the shared Selector's: its fields, field sets and security
9      return Database.query(newQueryFactory()
10       .setCondition('svc_OpenComplaintCount__c > :minimum').toSOQL());
11   }
12
13   public class Parameters implements ISelectorMethodParameterable {
14     public Integer minimumOpenComplaints;
15   }
16 }
17
18 // The caller, in service
19 List<Account> accounts = common_AccountsSelector.newInstance()
20   .selectInjection(svc_AccountsWithOpenComplaintsSelectorMethod.class, params);
```

Domain process injection

Story: Service reacts to a late shipment

common owns

The `Case` Domain, extending `ApplicationSObjectDomain`. Its trigger runs any injected processes for each trigger event

service ships

- **Criteria:** `svc_LateShipmentCriteria` (`IDomainProcessCriteria`) keeps complaint Cases about late orders
- **Action:** `svc_FlagAccountAction` (`IDomainProcessAction`) flags their Accounts

Two `DomainProcessBinding__mdt` records

- `RelatedDomainBindingSObject__c` = Case
- `TriggerOperation__c` = After_Insert
- `Type__c` = Criteria, then Action
- `OrderOfExecution__c` sets the order

Session 3 drew the line between the Domain and the trigger handler. Domain process injection is that same line, opened to other packages.

Platform event distribution

Story: Sales closes a deal, and the warehouse ships it

sales: when an Opportunity closes

```
AT4DXMessage__e evt = new AT4DXMessage__e();
evt.Category__c  = 'Opportunity';
evt.EventName__c = 'SALES_OPPORTUNITY_CLOSED_WON';
evt.Payload__c   = JSON.serialize(opportunityIds);
EventBus.publish(evt);
```

ful_ClosedWonConsumer.cls · fulfillment

```
public class ful_ClosedWonConsumer
  extends PlatformEventAbstractConsumer {
  public override void runInProcess() {
    Set<Id> oppIds = new Set<Id>();
    for (SObject sobj : events) {
      AT4DXMessage__e evt = (AT4DXMessage__e) sobj;
      oppIds.addAll((Set<Id>) JSON.deserialize(
        evt.Payload__c, Set<Id>.class));
    }
    ful_FulfillmentOrdersService.createFor(oppIds);
  }
}
```

fulfillment ships a `PlatformEvents_Subscription__mdt` record: `EventBus__c`, `EventCategory__c`, `Event__c`, `Consumer__c`. `PlatformEventDistributor` hands matching events to the consumer.

Events decouple the most

Pick the lightest tool that does the job

Why events

- The publisher doesn't know who listens
- No interface to keep stable, only an event name and payload
- Add a subscriber by installing a package

The trade-off

- Asynchronous: the deal closes before the order exists
- Harder to trace and to debug
- Document event names like a contract, because they are one

> Need an answer now? Use a contract (topic 2). Telling others something happened? Publish an event.

Meridian's stories, on the map

Every division works with the others, and no business package depends on another

Story	From → to	Technique	Lives in
Sales closes a deal, the warehouse ships it	sales → fulfillment	Platform event distribution	Subscription in fulfillment
Service asks where an order is	service → fulfillment	Contract + Service binding	Contract in common
Fulfillment schedules robot maintenance	fulfillment → maintenance	Contract + Service binding	Contract in warehouse-common
Marketing adds fields to Account	marketing → common	Selector field injection	Field set in marketing
Service reacts to a late shipment	service → common	Domain process injection	Criteria, action in service
Service queries its own Accounts	service → common	Selector method injection	Method class in service

Depend on contracts. Let configuration decide.

Promises kept

✅ Metadata-driven dependency injection	Andy, S2
✅ Interfaces to resolve Services	Andy, S2
✅ Selector fields from another package	Session 4
✅ `@NamespaceAccessible` : it's for ISVs	Session 4
✅ `WDGT_` , across real packages	Session 5
✅ AT4DX keeps a factory, on purpose	Session 5

Remember the map

Dependencies only point down.

Each object has one owner.

Push shared things down only as far as needed.

Thank you, Sally, Andy and John Storey, for the series.

Likely questions

Short answers to start from

Do I need AT4DX?

No. You can pass interfaces and use a small factory of your own. AT4DX pays off once several packages extend the same objects.

Migrating an org?

Pull shared objects and contracts into common first. Then move one feature at a time, replacing direct calls with factory calls.

Is `Application` required now?

Session 5 said it's optional in one package. Across packages, the factory is the point.

Harder to debug?

Somewhat. Clear binding names, a test that every binding resolves, and rare overrides.

Performance?

Custom metadata reads don't count against the SOQL limit. Load bindings once per transaction anyway.

Namespaces?

An ISV's concern with managed packages, and supported. Enterprises use unlocked packages and prefixes.

Learn more

Where to go next

- **AT4DX** and **AT4DX sample code** — github.com/apex-enterprise-patterns/at4dx and at4dx-samplecode
- **force-di** — github.com/apex-enterprise-patterns/force-di
- **fflib Apex Common** and **fflib Apex Mocks** — github.com/apex-enterprise-patterns/fflib-apex-common and fflib-apex-mocks
- **Brewtiful Beans Learns AT4DX** — a blog series by **Jake Richter** that walks through the same ideas with a coffee roaster: syntax-syllogism.com/blog/tags/brewtiful-beans
- **This series** — slides for all six sessions: github.com/ImJohnMDaniel/CodeWithSally-Apex-English

Thank you

Questions?

FFLib Series · Session 006

John M. Daniel

github.com/ImJohnMDaniel

